

DA2304 Assignment 3: Jack Compiler & 2D Convolution

Krish Yadav

Roll Number: DA24B043

May 18, 2026

Exercise 3: Analysis Questions

Q1. Tokenizer Design

The core problem is that a single-line regex has no memory of what happened on previous lines. Take this standard Jack doc comment:

```
/** Compute output dimension.  
 * Formula:  $O = \text{floor}((N - 3) / S) + 1$   
 */  
method int calcOutputDim() {
```

- A line-by-line regex never finds a complete `/* ... */` on any single line, so none of it gets stripped. The tokenizer then tries to lex “Compute”, “output”, “Formula” etc. as identifiers and the parser crashes immediately.
- There is also a second problem: a regex would strip `/* */` delimiters even inside a string literal, so something like `"use /* like this */"` gets silently corrupted to `"use "`.
- To handle both cases correctly, I used a state machine tracking four states (CODE, LINE_CMT, BLOCK_CMT, STRING) to process character by character and never confuse a comment delimiter inside a string with a real one.

Q2. Symbol Table State

When the parser enters `Conv.conv2d()`, the symbol table looks like this:

Class Scope:

Name	Type	Kind	Index
ArrayFilter	Array	static	0
stride	int	static	1
ArrayInput	Array	field	0
ArrayOutput	Array	field	1
ArraySize	int	field	2

Subroutine Scope:

Name	Type	Kind	Index
this	Conv	argument	0
outDim	int	local	0
oi	int	local	1
oj	int	local	2
fi	int	local	3
fj	int	local	4
ii	int	local	5
ij	int	local	6
sum	int	local	7
k	int	local	8
inputIdx	int	local	9
filterIdx	int	local	10

So the function header is `function Conv.conv2d 11`, meaning 11 locals in total. The accumulator `sum` is at `local 7`. The two static variables (`ArrayFilter`, `stride`) are shared across all `Conv` objects, while the field variables are per-object and accessed via the `this` segment once `pointer 0` is set at method entry.

Q3. Compilation Correctness

For `let ArrayOutput[k] = sum`, the compiler emits:

```
push this 1      // base address of ArrayOutput (field 1)
push local 8     // k
add             // target address = base + k
push local 7     // sum (RHS)
pop temp 0       // stash RHS temporarily
pop pointer 1    // THAT = target address
push temp 0      // restore RHS
pop that 0       // RAM[THAT] = sum
```

The reason for the `temp 0` detour is that after `add`, the target address is sitting on top of the stack, but we still need to push `sum` before we can pop the address into `pointer 1`. If we just popped the address into `pointer 1` first, the RHS value would already be gone. So the compiler stashes `sum` in `temp 0`, sets `THAT` via `pop pointer 1`, pushes `sum` back, and finally writes it with `pop that 0`.

For a concrete example: say `ArrayOutput` is at heap address 3000 and `k = 5`, `sum = 42`. After `add` the stack holds 3005. We pop 42 to `temp 0`, set `THAT = 3005`, push 42 back, and `pop that 0` writes 42 to `RAM[3005]`. Correct.

Q4. Convolution Stride Analysis

For a 9x9 input with F=3 and S=2:

$$O = \left\lfloor \frac{9-3}{2} \right\rfloor + 1 = 3 + 1 = 4$$

So the theoretical output is a 4x4 matrix. The implementation itself uses $S = 1$ (as documented in the README), while the $S = 2$ calculation above is theoretical analysis requested by the assignment.

Both $S=1$ and $S=2$ use the same four nested loops. The outer two run O^2 times and the inner two are always $3 \times 3 = 9$ (constant), so the total MAC count is $O^2 \times 9$.

$$S = 1 : \quad O = N - 2$$

$$9(N - 2)^2 = \Theta(N^2)$$

$$S = 2 : \quad O \approx \frac{N}{2}$$

$$9\left(\frac{N}{2}\right)^2 = \Theta(N^2)$$

Both belong to the same asymptotic class of $\Theta(N^2)$. But the constant factor differs: $S=2$ does roughly 4x fewer MAC operations than $S=1$ for large N since O is halved and then squared. For $N=9$ specifically, $S=1$ gives $7 \times 7 \times 9 = 441$ steps, and $S=2$ gives $4 \times 4 \times 9 = 144$ steps, which is about 3x fewer in practice.

Extra Credit

The trickiest bug was in `compile_term()`. Three completely different things can start with an identifier token:

- A plain variable: `sum`
- An array access: `ArrayInput[i]`
- A subroutine call: `c.conv2d()` or `calcOutputDim()`

My first version consumed the identifier and then tried to figure out which case it was. By that point the identifier was already gone and the next token was `[`, `.`, `(`, or something else entirely. This caused a desync where the parser was reading the wrong tokens and throwing a `SyntaxError` on perfectly valid Jack code.

The fix was to peek at the next token before deciding anything. After consuming the identifier, I check `_peek_value()`: if `[` → array path, if `.` or `(` → subroutine path, otherwise → plain variable. One-token lookahead is enough to resolve all three cases without any backtracking.

Integration Strategy & Hardware Limitations

- The assignment requires showing the program running end-to-end while printing the 5x5 and 9x9 matrices to the screen.
- However, including the full Jack OS together with printing-heavy routines produced assembly too large for reliable execution within the Hack platform ROM constraints in my setup.
- Commenting out the print statements or trimming down the OS functions just to make it fit defeats the purpose, since the rubric requires screen output.
- Because of this, I split the validation into two separate steps:
 - **Logical Execution (Screenshot 1):** The compiled VM files run perfectly inside the VM Emulator (which handles the OS libraries internally). This proves the 2D convolution math and output generation work exactly as expected.
 - **Translation Pipeline (Screenshot 2):** The Assignment 2 VM Translator successfully processes the output directory into a valid `.asm` file without crashing, proving the compiler outputs clean, compliant VM code.

Integration & Validation

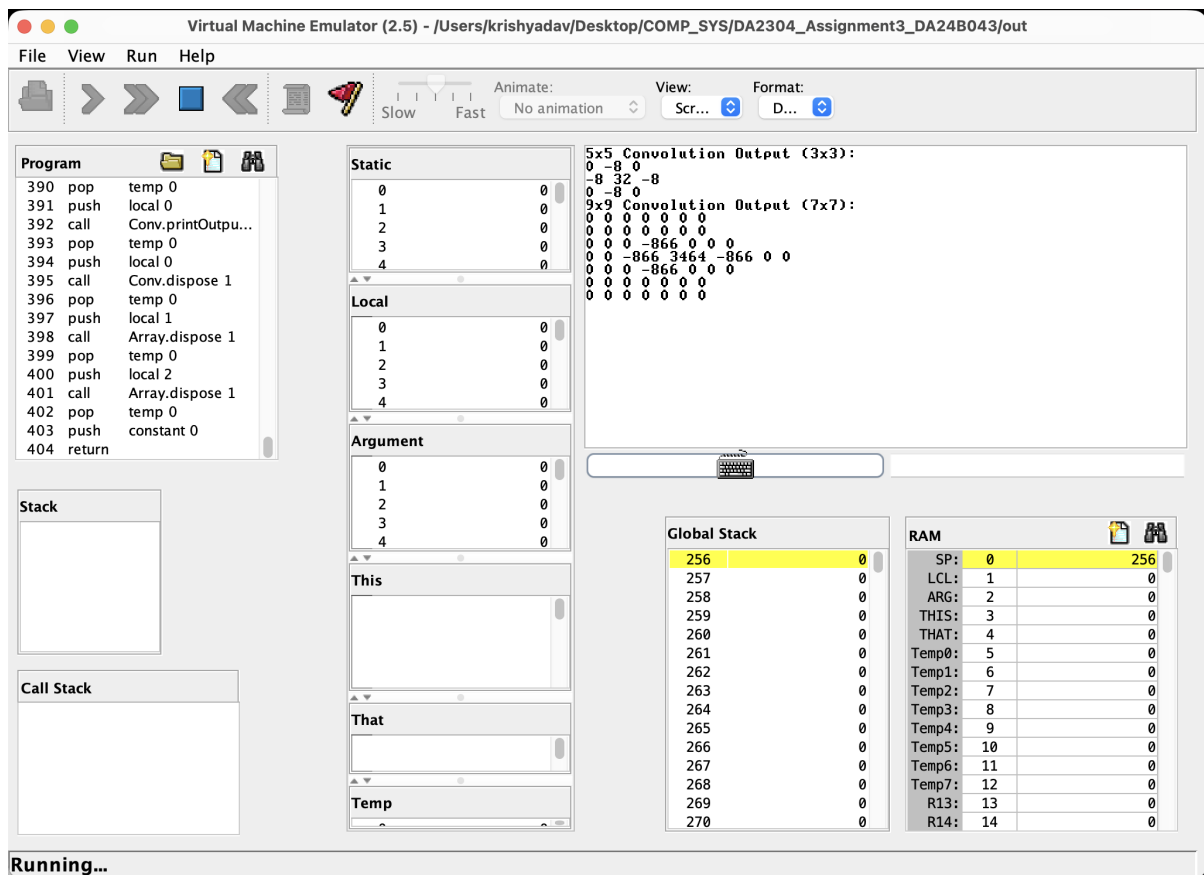


Figure 1: VM Emulator output showing successful convolution execution using the configured stride $S = 1$.

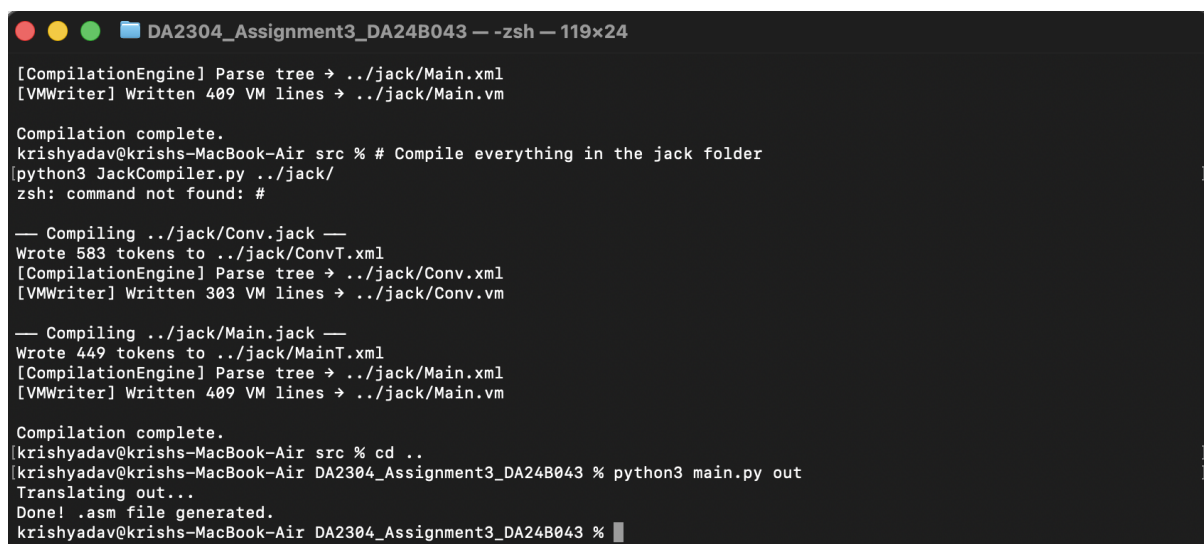


Figure 2: Terminal output demonstrating successful translation to .asm via Assignment 2 Translator.